

Visualytica

Project Team

Muhammad Umar	21I-1787
Syed Saad Khurshid	21I-1760
M Owais Zahid	21I-1709

Session 2021-2025

Supervised by

Dr. Faisal Cheema



Department of Data Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Problem Statement	2
1.3	Scope	2
1.4	Modules	3
1.4.1	Module 1: Web Design/ Plots/Area graph	3
1.4.2	Module 2: End-to-end Dynamic Dashboard & Summing Up	3
1.5	User Classes and Characteristics	3
2	Visuallytica: Simplified Graph Visualization	5
2.1	Use-case/Event Response Table/Storyboarding	5
2.1.1	Use Case	5
2.1.2	Event-Response Table	6
2.1.3	Storyboarding	7
2.2	Functional Requirements	7
2.2.1	Functional Requirements of Visuallytica	8
2.2.2	Module 1	9
2.2.3	Module 2	9
2.3	Non-Functional Requirements	9
2.3.1	Reliability	9
2.3.2	Usability	10
2.3.3	Performance	10
2.3.4	Security	10
2.4	Domain Model	11
3	System Overview	12
3.1	Architectural Design	12
3.1.1	Diagrams	13
3.2	Design Models	14
3.2.1	Procedural Design Models	15
3.2.1.1	Activity Diagram	15
3.2.1.2	System Sequence Diagram	15
3.2.1.3	Data Flow Diagram	16

3.3	Data Design	17
3.3.1	User Data	17
3.3.2	Graph Data	17
3.3.3	Data Structures	17
3.3.4	Database and Storage	18
4	Conclusion	19

List of Figures

2.1	Use Case Diagram for Visualytica	6
2.2	Domain Model for Visualytica	11
3.1	Box and Line Diagram of System Architecture	13
3.2	Finalized Architecture Style: Multi-tiered Client-Server	14
3.3	Component Interaction Diagram	14
3.4	Activity Diagram for Visualytica	15
3.5	System Sequence Diagram for Visualytica	16
3.6	Data Flow Diagram for Visualytica	16

List of Tables

1.1	User Classes and Their Characteristics	4
2.1	Event-Response Table for Visualytica	7

Chapter 1

Introduction

1.1 Introduction

This document talks about an app called **Visualytica** that helps make graphs easy to understand and use. The app is for people who need to see data in graph form but don't want to deal with complicated steps.

Here's who might read this document:

- **Developers:** People who will build or code the app.
- **Project Managers:** People who organize the project and make sure everything is going as planned.
- **Marketing Staff:** People who will tell others about the app and get people to use it.
- **Users:** People who will use the app to make or view graphs.
- **Testers:** People who check if the app works correctly.
- **Documentation Writers:** People who write instructions on how to use the app.
- **FYP Panel:** The group of professors and evaluators who will assess the project's quality, functionality, and its contribution to solving the problem it was designed for.

?

1.2 Problem Statement

Data visualization is essential for businesses, especially today. Existing tools for graph data (nodes and edges) are often costly, complex, and lack flexibility for specific visualization needs. Users must manually transform raw graph data into insights, a process that can be time-consuming and prone to errors.

These tools require extensive coding, making them difficult for non-programmers and even technical users due to their steep learning curves. This limits the broader application of graph visualization, hindering users from fully leveraging their data.

Visuallytica addresses these issues by simplifying graph visualization. It reduces technical complexity and improves interactivity, enabling users to easily analyze large datasets and extract meaningful insights.

1.3 Scope

Visuallytica is a development project aimed at the creation of a dedicated web platform for visual analysis on graph databases. The platform will also allow users to build sophisticated graph (node and edge) visualizations with an intuitive interface for customization and interaction.

Visuallytica will embed a concise yet powerful recommendation engine, customized to graph visualizations. The algorithm will help you make sense of the structure in your graph data so it can suggest which visualizations are most appropriate. Or, if the graph is a tree it will recommend using trees for visualization. It also recommends layouts like force-directed graphs or hierarchical visualizations for more complex networks. The recommendation feature will help users choose the most intelligent visuals for their data.

The platform will also get in build support for interactive possibilities like navigating through the graph structures, connections and patterns on-the-fly. By concentrating only on graph visualization *Visuallytica* streamlines the complex networks comprehension providing non-technical users with actionable insights.

The portal will also include basic dashboard capabilities, allowing for multiple chart views to be together into a unifying interactive view of your data. In addition, the platform will collate using a domain specific chatbot that informs users over visualization concepts such as how to comprehend and expose connection graph data.

1.4 Modules

The Visuallytica project is built up in two main modules, designed to deliver the most important functionalities for a complete data visualization platform.

1.4.1 Module 1: Web Design/ Plots/Area graph

It helps us understand how to design a web interface for generating and customizing plots in the next module. The intent is that this would become a user-friendly, responsive web layer for users to create and maintain their own views.

1. **Web Design, Development & backend data processing** : Create an interactive layout to meet all new future trends.helper classes for color and background images. This includes input elements for loading data, choosing a visualization and an interface to the recommendation functionality, that recommends plot types based on user provided dataset.
2. **Graph generation & Customization**: Develop a few algorithms to generate graph-based visualisations—tree graphs, (directed) network graphs; allow users to customise various graphical elements like node size/edge thickness/color/label/layout-points

1.4.2 Module 2: End-to-end Dynamic Dashboard & Summing Up

In this module, we will create a dynamic dashboards and add the final integration in testing of our platform.It ensures everything sews together nicely, and meets the standard of quality anticipated.

1. **Generation of Dynamic Dashboards**: Create the system able to build interactive and dynamic dashboards combining multiple visual outputs with interactive features such as filters and drilldowns.
2. **Testing and Final Integration**: Testing of all modules, fixing problems if any, and completing the integration of all the components with the aim to execute the entire platform.

1.5 User Classes and Characteristics

The following table identifies the different types of users who are expected to interact with the Visuallytica app. Each user class has specific needs and characteristics that influence

how they will use the product. Understanding these user classes will help ensure that the app meets the requirements of a diverse audience

User Class	Description
Developers	Developers are responsible for building and maintaining the Visuallytica app. They will interact with the code, design features, and fix any bugs or issues. They need detailed documentation and flexibility to implement new features.
Project Man- agers	Project Managers oversee the progress of Visuallytica, ensuring that development stays on schedule and meets key milestones. They require high-level overviews of the project and regular updates on progress.
Marketing Staff	Marketing Staff are responsible for promoting Visuallytica to potential users. They need a clear understanding of the app's features, target audience, and selling points to effectively communicate its value.
Users	Users will be the primary audience for the app, utilizing it to visualize or create graphs. They need a user-friendly interface and easy access to all features without requiring deep technical knowledge.
Testers	Testers are tasked with checking the app for bugs, ensuring that Visuallytica functions as intended. They will need access to all parts of the app for comprehensive testing and reporting issues to developers.
FYP Panel	The FYP Panel, made up of academic evaluators, will assess the quality and impact of Visuallytica. Their focus will be on the technical implementation, innovation, and usability of the app in solving the problem it is designed for.

Table 1.1: User Classes and Their Characteristics

Chapter 2

Visualytica: Simplified Graph Visualization

This chapter outlines the functional and non-functional requirements of Visualytica, a web-based tool designed to simplify graph visualization and enhance user interactivity.

2.1 Use-case/Event Response Table/Storyboarding

The selection of requirement gathering techniques depends on the nature of the project. For Visualytica, we have chosen the following methods to represent the system's functionalities:

2.1.1 Use Case

A use case is a suitable technique for interactive, end-user applications like Visualytica. It helps define how users will interact with the system and what goals they can achieve. Below are some use cases for Visualytica:

- **Create Graph:** The user selects the input format and provides data to generate a graph.
- **Interact with Graph:** The user can zoom, pan, and view detailed information about specific nodes.
- **Perform Graph Analysis:** The user requests graph analysis, such as shortest paths or node centrality.
- **Export Graph:** The user exports the graph as an image (e.g., PNG, SVG).

- **Upload Data and Suggest Graphs:** The user uploads the dataset, and the system automatically detects the data type, suggesting suitable graph layouts. The user can also manually select graph types.
- **Design Dashboards and Customize Plots:** Users can design interactive dashboards, customize plot types, colors, and styles, and display multiple graphs simultaneously.

Use Case Diagram:

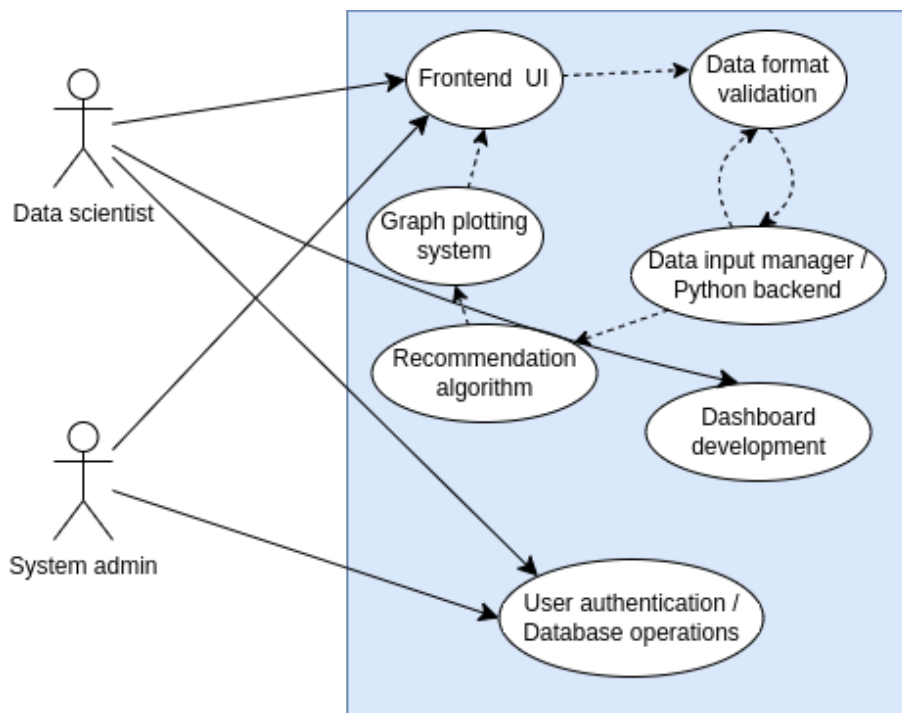


Figure 2.1: Use Case Diagram for Visuallytica

2.1.2 Event-Response Table

This table is suitable for real-time systems where most actions are handled in the background. Visuallytica's backend processes tasks like graph analysis, ensuring that the frontend remains responsive.

Event	Response
User uploads dataset	System detects the data type and suggests appropriate graph layouts
User selects or customizes graph	System generates and displays the selected graph layout in real-time
User requests analysis	System processes the request and highlights results on the graph
User exports graph	System generates a downloadable image (PNG, SVG)
User designs a dashboard	System creates and displays an interactive dashboard with multiple graphs

Table 2.1: Event-Response Table for Visuallytica

2.1.3 Storyboarding

Storyboarding is ideal for applications with a strong visual focus, like Visuallytica, where users interact with graphs. Below is a brief outline of the steps users will follow:

- **Graph Format Selection:** The user chooses the input format (adjacency matrix, edge list, or adjacency list).
- **Data Input:** The user provides graph data either manually or through file upload.
- **Graph Suggestions:** The system suggests graph layouts based on the detected data type, or the user can manually select the graph type.
- **Graph Visualization:** The system generates an interactive graph based on the selected layout.
- **Dashboard Design and Customization:** Users design an interactive dashboard, customizing graphs and adding multiple plots.
- **Export:** The user exports the graph or dashboard as an image (e.g., PNG, SVG).

2.2 Functional Requirements

This section outlines the functional requirements of the **Visuallytica** system. Each feature of the system is described in detail, followed by specific functional requirements that the system must meet. These requirements define the core behavior of the system, providing

a clear specification of what the system must accomplish to satisfy the needs of its users. The features are organized in a structured manner to cover all aspects of the system's functionality, ensuring that Visuallytica operates efficiently, accurately, and as intended.

2.2.1 Functional Requirements of Visuallytica

1. Data Upload and Input Formats

- **FR 1.1:** The system shall support multiple graph data input formats, including Adjacency Matrix, Edge List, and Adjacency List.
- **FR 1.2:** Users shall be able to upload datasets via a user-friendly interface supporting file formats such as CSV and JSON.
- **FR 1.3:** The system shall validate the structure of uploaded datasets to ensure compliance with the required formats.
- **FR 1.4:** Users shall be provided with error messages and instructions if an invalid file format is uploaded.

2. Data Visualization

- **FR 2.1:** The system shall visualize graph data in various formats such as directed and undirected graphs.
- **FR 2.2:** Users shall be able to interact with the graphs, including zooming, panning, and clicking nodes to explore details.
- **FR 2.3:** The system shall allow users to apply filters on the graph to customize the view based on node or edge attributes.

3. Dashboarding

- **FR 3.1:** The system shall provide a dashboard that offers users an overview of key metrics and statistics related to their datasets.
- **FR 3.2:** The dashboard shall display visual summaries such as graphs, charts, and tables that represent the structure and properties of the uploaded graph data.
- **FR 3.3:** Users shall be able to customize the dashboard by selecting which metrics and data visualizations to display.
- **FR 3.4:** The system shall automatically update the dashboard in real time as new data is uploaded or changes are made to the graph.

2.2.2 Module 1

Following are the requirements for module 1:

1. **M1FR1:** Web implementation and backend data processing/formatting for validating or analyzing uploaded data to detect whether the input is a tree, Directed Acyclic Graph (DAG), or simple graph.
2. **M1FR2:** Generation and customization of all graph types, including trees, DAGs, and other graph structures.

2.2.3 Module 2

Following are the requirements for module 2:

1. **M2FR1:** Design of dynamic dashboards where users can customize the layout and presentation of graphs and visualizations.
2. **M2FR2:** Final frontend and backend integration, including comprehensive testing to ensure system functionality and performance.

2.3 Non-Functional Requirements

This section explains the non-functional requirements for Visualytica. These requirements focus on how well the system should work and are specific, measurable, and easy to verify.

2.3.1 Reliability

For Visualytica, reliability means how long the system can operate without crashing or producing errors. The system should have a high Mean Time Between Failures (MTBF), meaning it should work for a long time without breaking. If something does go wrong, Visualytica should recover quickly and protect the user's data.

Example:

- **REL-1:** Visualytica should maintain an uptime of at least 99.5%, meaning it is operational for all but 0.5% of the time in a month.
- **REL-2:** If a failure occurs, the system should automatically save the user's work and allow recovery within 30 seconds.

2.3.2 Usability

Visuallytica should be easy for users to understand and use, even if they are not experts in graph theory. The interface should be user-friendly, allowing users to easily upload data, generate graphs, and customize visualizations without needing a lot of training. The system should also provide help features, including error messages that guide users to fix issues.

Example:

- **USE-1:** Users should be able to generate a graph from their data within 3 steps (uploading, selecting a graph format, and viewing the graph).
- **USE-2:** The system should offer tooltips or tutorials that explain the various graphing options for first-time users.

2.3.3 Performance

Performance is about how fast and responsive Visuallytica is when users interact with the system. The system should handle large datasets and complex graph algorithms quickly without causing long delays.

Example:

- **PER-1:** Visuallytica should load any graph visualization within 5 seconds for datasets up to 10000 nodes and edges.
- **PER-2:** Visuallytica should allow users to zoom, pan, and interact with graphs without noticeable lag.

2.3.4 Security

Visuallytica must ensure that users' data is protected and secure. Only authorized users should have access to their graphs, and data should be encrypted. The system should also protect against unauthorized access and ensure data privacy.

Example:

- **SEC-1:** All user data in Visuallytica should be encrypted during both storage and transmission.
- **SEC-2:** The system should require user authentication for accessing saved graphs or dashboards.

- **SEC-3:** Visuallytica should log any unsuccessful attempts to access user data, alerting administrators of potential security threats.

Here's an introductory heading for your Domain Model section, followed by instructions to insert the diagram:

2.4 Domain Model

The domain model represents the key entities and their relationships in the Visuallytica system. It captures the conceptual structure of the system, illustrating how different components such as users, datasets, visualizations, and graphs interact with each other. This model provides a foundation for understanding the system's design, guiding the development process by defining the data and operations required.

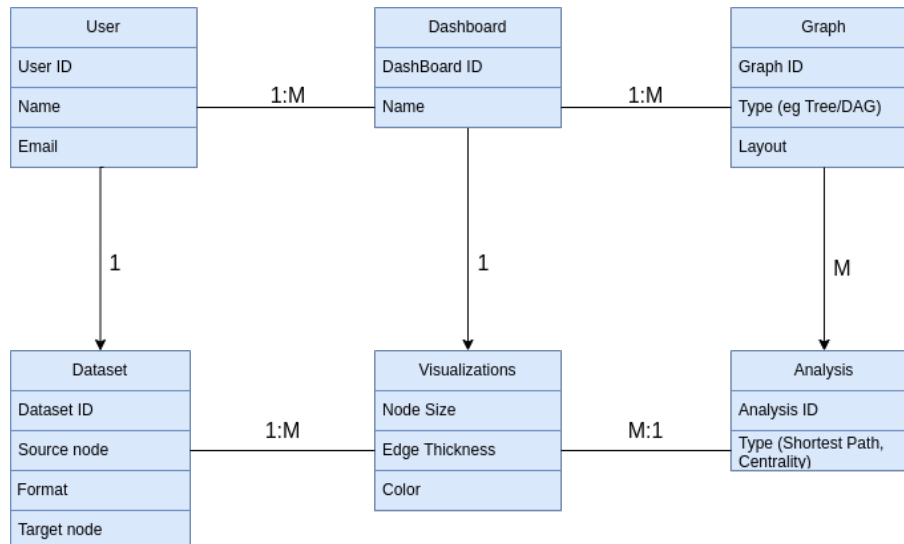


Figure 2.2: Domain Model for Visuallytica

Chapter 3

System Overview

This chapter provides an in-depth description of the architecture and design of Visualytica, focusing on the system's high-level structure and its underlying components. The architecture is designed to support scalable, interactive graph visualizations and ensure smooth integration between various system components.

3.1 Architectural Design

In this section, we provide a high-level overview of the system's architecture and explain how its main modules collaborate to achieve the full functionality of Visualytica. The system follows a modular design approach, where each module is responsible for a specific task, making the system easier to maintain and extend.

The major subsystems include the following:

- **Frontend Module:** Responsible for handling user interactions and rendering the graph visualizations. It allows users to input data in different formats, configure the graph properties, and display the results.
- **Backend Module:** This module processes the data received from the frontend, performs graph-related operations, and communicates with the database for storing user projects and configurations.
- **Graph Processing Module:** The core engine of the system, responsible for creating, manipulating, and analyzing graphs. It supports multiple input formats such as adjacency matrices, edge lists, and adjacency lists.
- **Database Module:** Manages user authentication by storing and verifying user credentials using a NoSQL database like MongoDB.

These subsystems work together to ensure smooth data flow and provide users with an intuitive platform for visualizing graphs.

3.1.1 Diagrams

To provide a clearer understanding of how the system is structured, we will present the following diagrams:

- **Box and Line Diagram:** A simple representation of the overall system architecture, showing the main modules and how they interact.
- **Finalized Architecture Diagram:** A detailed diagram mapping the modules to an architecture style. For this system, we follow a multi-tiered client-server pattern.
- **Component Interaction Diagram:** Illustrates how the frontend, backend, and graph processing modules work together during key operations.

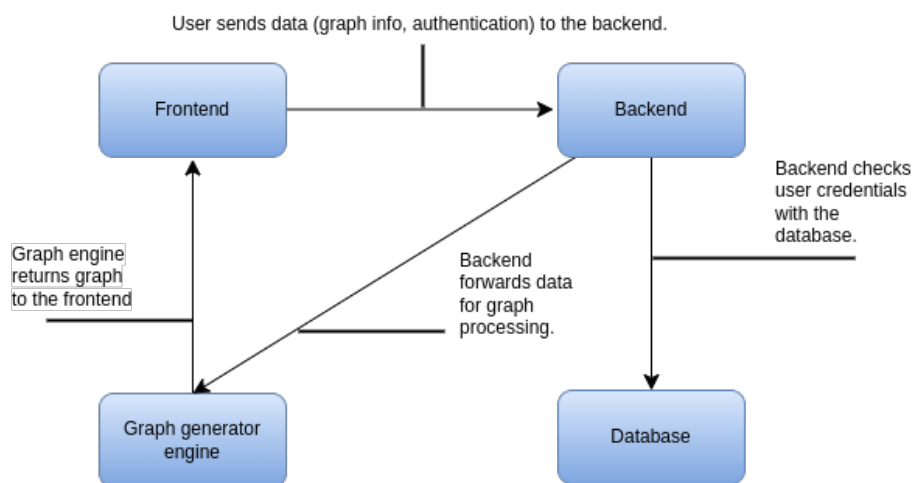


Figure 3.1: Box and Line Diagram of System Architecture

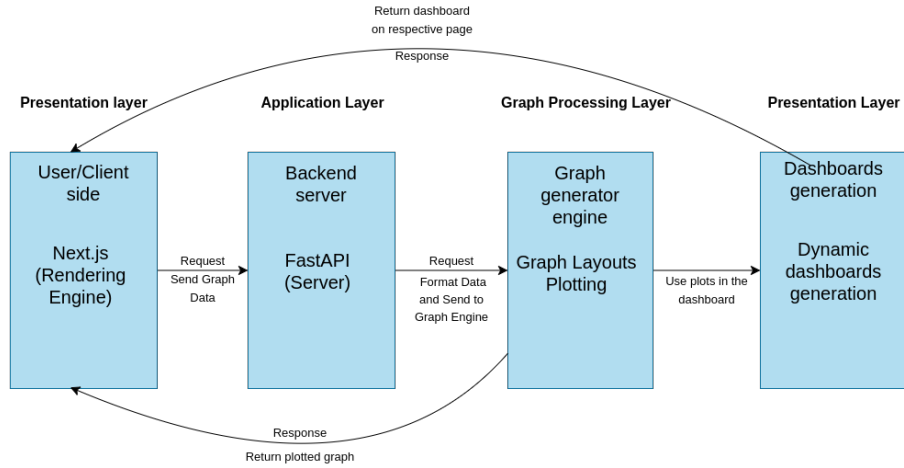


Figure 3.2: Finalized Architecture Style: Multi-tiered Client-Server

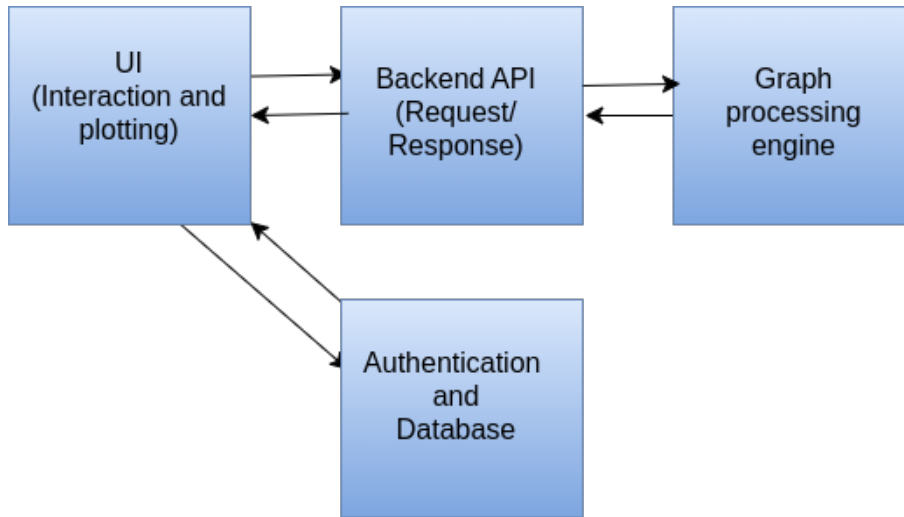


Figure 3.3: Component Interaction Diagram

The diagrams above demonstrate how Visuالتica is designed as a modular system, ensuring flexibility, scalability, and maintainability.

3.2 Design Models

In this section, we will present the design models for Visuالتica, using the procedural approach. These models will help us understand how the system works by showing the flow of data and the steps involved in processing and displaying graph visualizations.

We will create different types of diagrams to explain the system's functionality. These diagrams will provide a clear picture of how the different parts of Visuالتica interact and work together.

3.2.1 Procedural Design Models

Visualytica uses a procedural approach to break down the tasks and processes in the system. Below are the design models that apply to this project.

3.2.1.1 Activity Diagram

The Activity Diagram shows the sequence of actions a user or system performs. It helps us understand the flow of activities from the user interface to the backend and how the data is processed and returned.

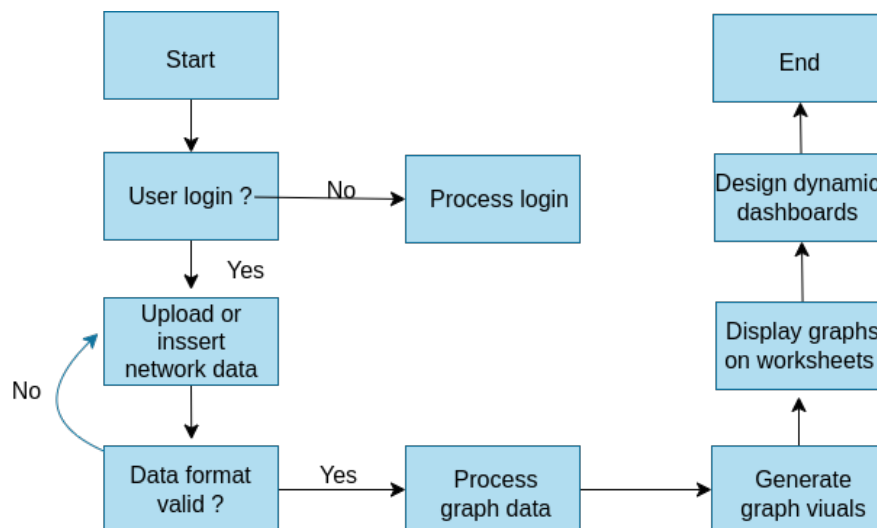


Figure 3.4: Activity Diagram for Visualytica

3.2.1.2 System Sequence Diagram

The System Sequence Diagram shows the interaction between the user and the system during key processes, like submitting graph data and receiving the visualization. It helps illustrate the order of messages passed between the user, frontend, backend, and graph generation engine.

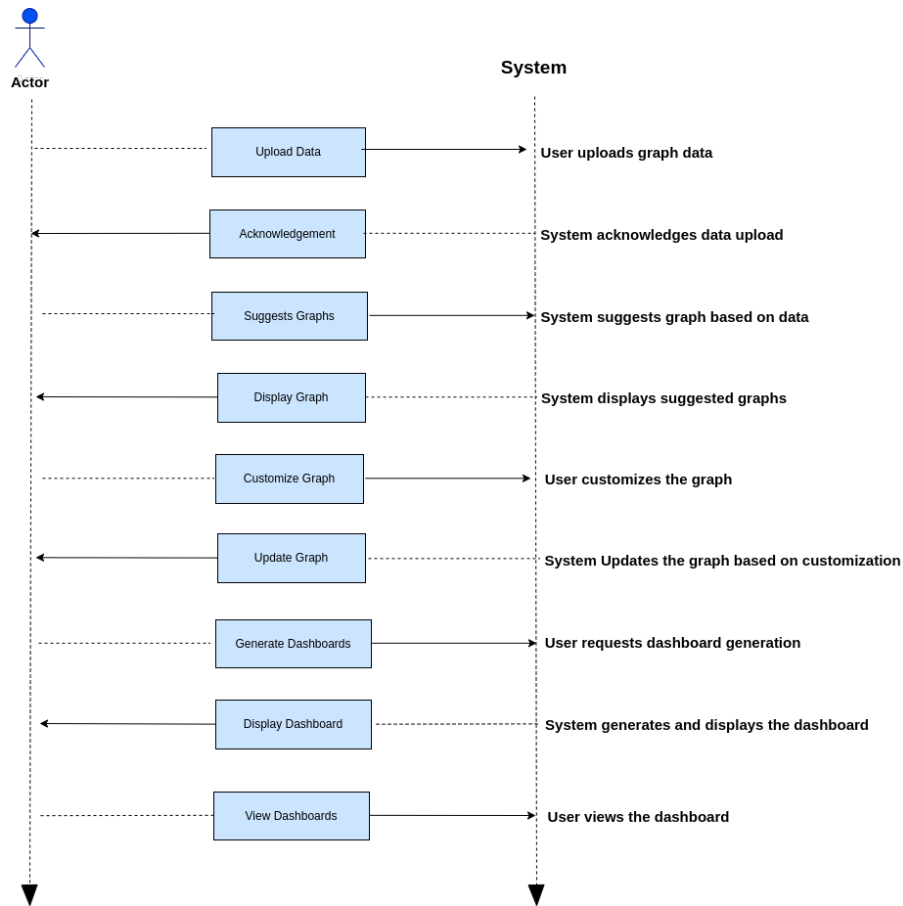


Figure 3.5: System Sequence Diagram for Visuallytica

3.2.1.3 Data Flow Diagram

The Data Flow Diagram (DFD) shows how data moves through the system. It includes sources of data, processes that transform the data, and where the data is stored. For Visuallytica, this diagram will detail the flow of graph data from input to visualization.

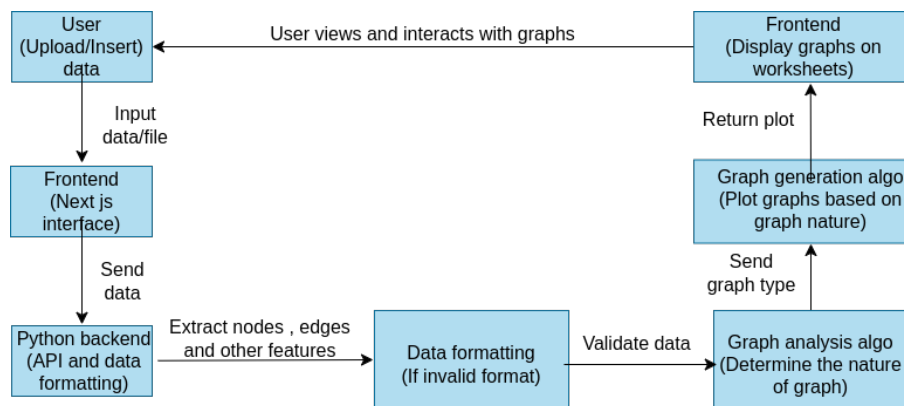


Figure 3.6: Data Flow Diagram for Visuallytica

These design models help us break down the system into simple steps, making it easier to understand and implement.

3.3 Data Design

In this section, we explain how the information from users and graphs is turned into data that the system can use. We'll describe how the data is stored, processed, and organized to make the system work smoothly.

3.3.1 User Data

User-related information, such as login details and authentication, is handled through a MongoDB database. MongoDB stores all the user data securely, like usernames and passwords, ensuring that only the correct users can log in and interact with the system.

3.3.2 Graph Data

The graph data, such as nodes, edges, and graph types (like trees, DAGs, or simple graphs), is stored in different formats depending on user input. These formats are supported to allow flexible input and processing:

- **Adjacency Matrix:** This format uses a matrix to show which nodes are connected. Rows and columns represent nodes, and the matrix cells contain values indicating whether there is an edge between nodes. This format is useful for analyzing dense graphs.
- **Edge List:** The edge list format represents the graph as a list of all the edges, where each entry shows a pair of connected nodes. This format is efficient for sparse graphs and makes it easy to validate and plot graphs.
- **Adjacency List:** In the adjacency list format, each node has a list of its neighboring nodes. This structure is lightweight and works well for storing large graphs with fewer connections.

3.3.3 Data Structures

Several important data structures are used to handle and process the graph data:

- **Adjacency Matrix:** Stored as a 2D array, where each row and column represents a node, and values in the matrix indicate whether a connection exists between nodes.
- **Edge List:** A list of edges where each element is a pair (or triple, if weighted) of nodes. This format is ideal for graph visualization and validation tasks.
- **Adjacency List:** A dictionary or list of lists, where each key (or index) represents a node, and the associated value is a list of all the nodes it's connected to.
- **Node and Edge Attributes:** Any extra information about the nodes (like labels or weights) is stored in dictionaries, making it easy to access this data when needed.

3.3.4 Database and Storage

- **MongoDB:** MongoDB is used only for handling user authentication. It stores user accounts and passwords in a secure manner.
- **Backend File Storage:** The graph data uploaded by the user is stored in the back-end's /data directory. It is organized into different files based on user input, but no user progress is saved for now.

Chapter 4

Conclusion

So far, we've completed some important parts of both the frontend and backend for Visualyca. The prototype UI is up and running, and we've developed the landing page and the workspace page. Our backend processing is functional, but we still need to improve it by allowing more flexibility in the data formats that can be uploaded. Overall, our development pace has been good, and we're working to make it even faster in the next phase. Future efforts will focus on refining the backend and adding more flexibility and functionality to the app.